

GNNHAR-IV 资产规模扩展实验方案：从 Dow 30 到 S&P 100 / S&P 500

Volatility Project Design Note

2026 年 6 月 10 日

1 核心研究问题

你现在这篇文章的主问题可以自然扩展成一个更强的 scale-effect 问题：

当资产数量 N 从 30 增加到 100 或 500 时，图结构带来的预测增益是否变大？

更具体地说，不只问 GNNHAR-IV 是否比 HAR-IV 好，而是估计下面这些增益如何随 N 变化：

$$\Delta_N^{GHAR} = L(HAR_N) - L(GHAR_N),$$

$$\Delta_N^{GNN} = L(GHAR_N) - L(GNNHAR_N),$$

$$\Delta_N^{GNN-IV} = L(GHAR-IV_N) - L(GNNHAR-IV_N),$$

其中 $L(\cdot)$ 可以分别取 test MSE 或 test QLIKE。论文中最重要的表述不是“某一个模型赢了”，而是

$$\Delta_{100}^{GNN-IV} > \Delta_{30}^{GNN-IV} \quad \text{或} \quad \Delta_{500}^{GNN-IV} > \Delta_{100}^{GNN-IV}.$$

这才直接回答“更大的图是否让图神经网络更有用”。

2 当前项目的数据口径

当前 Dow 30 pipeline 使用三张宽表：

文件	形状	含义
merged_rv_data_filled.csv	Date $\times$ N	目标变量 $v_{i,t}$ ，项目里的 realized volatility / historical
merged_iv_data_filled.csv	Date $\times$ N	隐含波动率 $x_{i,t}$ ，当前使用 AlphaQuery 30-day IV Me
dow30_daily_returns_2021_2026.csv	Date $\times$ N	日收益率 $r_{i,t}$ ，用于 GLASSO 图结构

代码当前采用重叠 HAR 窗口：

$$H_{i,t-1}^{RV} = \left(v_{i,t-1}, \frac{1}{5} \sum_{\ell=1}^5 v_{i,t-\ell}, \frac{1}{22} \sum_{\ell=1}^{22} v_{i,t-\ell} \right),$$

$$H_{i,t-1}^{IV} = \left(x_{i,t-1}, \frac{1}{5} \sum_{\ell=1}^5 x_{i,t-\ell}, \frac{1}{22} \sum_{\ell=1}^{22} x_{i,t-\ell} \right).$$

扩展到 S&P 100 或 S&P 500 时，最重要的是保持同一口径：

- $v_{i,t}$ 不要一部分来自 LOBSTER 5-minute realized variance，另一部分来自 AlphaQuery historical volatility。
- $x_{i,t}$ 统一用 AlphaQuery 30-day IV Mean，必要时再加 call IV、put IV、skew 等扩展实验。
- $r_{i,t}$ 用同一调整价格源计算，例如 Yahoo Finance adjusted close 的 log return。
- 时间范围固定，例如 2021-01-26 到 2026-01-26，所有 universe 取共同交易日。

3 Universe 设计

建议跑四个 universe，而不是只跑 30 和 500：

Universe	目的	选择方法
Dow 30	当前基准	保留现有结果，作为 $N = 30$ 的 anchor
S&P 100	对齐 Zhang et al. 的大致规模	用 S&P 500 中市值最大且 IV/RV 完整的 100 只
S&P 200	中间规模检验	用同一排序取前 200 只，观察增益曲线是否单调
S&P 500-clean	最大规模检验	使用数据完整性过滤后的 S&P 500 成分股

不建议直接说“全部 S&P 500”而不筛选，因为 AlphaQuery IV 很可能存在缺失、换 ticker、并购退市、指数成分历史变化等问题。论文里应定义为：

$$\mathcal{U}_{500}^{clean} = \{i \in \text{S\&P 500 current constituents} : \text{RV, IV, return coverage pass filters}\}.$$

推荐过滤规则：

- 时间覆盖率：每只股票在样本期内 $v_{i,t}$ 、 $x_{i,t}$ 、 $r_{i,t}$ 的非缺失比例至少 95%。
- IV 极端值：winsorize 到每只股票 1% 和 99% 分位，或全局合理区间。
- 股票数量固定：先筛出 clean universe，再按市值排序取 $N = 30, 100, 200, 500$ 。
- Dow 30 作为单独结果保留；S&P 30 可以作为 robustness，因为 Dow 30 与 S&P top 30 不是同一组股票。

4 数据来源方案

4.1 收益率

收益率最容易：

$$r_{i,t} = \log(P_{i,t}^{adj}) - \log(P_{i,t-1}^{adj}).$$

可用 Yahoo Finance 或 Stooq 获取 adjusted close。收益率只用于 GLASSO，不直接作为目标。

4.2 IV

沿用你当前 paper 的 AlphaQuery 口径：

$$x_{i,t} = \text{AlphaQuery 30-day Implied Volatility Mean.}$$

实际采集时每个 ticker 保存一份原始文件，再合并成宽表：

```
data/raw/alphaquery_iv/AAPL.csv
data/raw/alphaquery_iv/MSFT.csv
...
data/processed/sp500/merged_iv_data_filled.csv
```

每个 ticker 的原始表至少保留：

```
Date,ticker,iv_mean_30,iv_call_30,iv_put_30,
put_call_iv_ratio_30,iv_mean_skew_30,
historical_volatility_30
```

主实验只用 `iv_mean_30`。其他字段留给 appendix。

4.3 RV / 目标变量

这里有两个可选路线，必须二选一。

路线 A：和当前项目最一致。 继续使用 AlphaQuery 或同源数据里的 `historical/realized volatility` 字段作为 $v_{i,t}$ 。优点是能覆盖 S&P 500，和你当前 Dow 30 文件最接近。缺点是严格意义上不是自己从高频价格计算的 `realized variance`。

路线 B：更严格但更贵。 从高频 5-minute returns 构造 `realized variance`：

$$RV_{i,t} = \sum_{m=1}^M r_{i,t,m}^2.$$

优点是更贴近 Zhang et al. 的 `realized-volatility literature`。缺点是 S&P 500 高频数据成本和清洗工作量很大。

当前建议：主实验用路线 A，把变量名在论文里谨慎写成项目中的 `volatility target` $v_{i,t}$ ，不要过度声称是高频 `realized variance`；如果未来拿到高频数据，再做路线 B 的 `robustness`。

5 模型组设计

每个 universe 都跑同一套模型：

模型	输入	用途
HAR	$H_{i,t-1}^{RV}$	无图、无 IV 的基准
GHAR	$H_{i,t-1}^{RV}, (WH_{t-1}^{RV})_i$	线性图结构增益

模型	输入	用途
GNNHAR1L/2L/3L	H_{t-1}^{RV}, W	非线性、多跳图增益
HAR-IV	$H_{i,t-1}^{RV}, H_{i,t-1}^{IV}$	IV 信息基准
GHAR-IV	$H^{RV}, H^{IV}, WH^{RV}, WH^{IV}$	线性图和 IV 的互补性
GNNHAR-IV	$[H^{RV}, H^{IV}], W$	主要研究模型
HAR-fakeIV	$H^{RV}, \tilde{H}^{IV}$	IV 参数扩张 placebo
GHAR-fakeIV	$H^{RV}, \tilde{H}^{IV}, WH^{RV}, W\tilde{H}^{IV}$	线性 placebo
GNNHAR-IV-fakeIV	$[H^{RV}, \tilde{H}^{IV}], W$	神经网络 placebo
GHAR-random / GNNHAR-random	使用 density-matched random W	图结构 placebo

其中 fake IV 用日期置换：

$$\tilde{H}_{i,t}^{IV} = H_{i,\pi(t)}^{IV}.$$

它保留 IV 的尺度和维度，但破坏时间预测关系。

6 图结构

训练期收益率矩阵记为 R_t 。GLASSO 估计 precision matrix：

$$\hat{\Theta} = \arg \min_{\Theta \succ 0} \{ \text{tr}(S\Theta) - \log \det(\Theta) \lambda \|\Theta\|_1 \}.$$

构造邻接矩阵：

$$A_{ij} = \mathbf{1}\{|\hat{\Theta}_{ij}| > \tau, i \neq j\}.$$

归一化：

$$W = D^{-1/2} A D^{-1/2}.$$

S&P 500 规模下 GLASSO 会更重，建议：

- 先在 S&P 100 和 S&P 200 调好 λ 或 edge density。
- S&P 500 用固定目标密度，例如 5%、10%、15%，避免图太密导致 WH 变成市场平均。
- 主表报告选定图；appendix 报告不同 density 的 robustness。

7 评估指标

测试集 MSE：

$$MSE = \frac{1}{|\mathcal{T}_{test}|N} \sum_{t \in \mathcal{T}_{test}} \sum_{i=1}^N (v_{i,t} - \hat{v}_{i,t})^2.$$

测试集 QLIKE：

$$QLIKE = \frac{1}{|\mathcal{T}_{test}|N} \sum_{t \in \mathcal{T}_{test}} \sum_{i=1}^N \left(\frac{v_{i,t}}{\hat{v}_{i,t}} - \log \frac{v_{i,t}}{\hat{v}_{i,t}} - 1 \right).$$

核心结果要报告 loss ratio:

$$\rho_{m,N}^{HAR} = \frac{L(m, N)}{L(HAR, N)}, \quad \rho_{m,N}^{HAR-IV} = \frac{L(m, N)}{L(HAR-IV, N)}.$$

再报告图结构增益:

$$G_N^{linear} = 1 - \frac{L(GHAR_N)}{L(HAR_N)},$$

$$G_N^{gnn} = 1 - \frac{L(GNNHAR_N)}{L(GHAR_N)},$$

$$G_N^{gnn-iv} = 1 - \frac{L(GNNHAR-IV_N)}{L(GHAR-IV_N)}.$$

如果 G_{100}^{gnn-iv} 或 G_{500}^{gnn-iv} 明显大于 G_{30}^{gnn-iv} , 你的 scale hypothesis 才有强证据。

8 代码组织

建议新增以下目录, 而不是继续把所有逻辑塞进 Dow 30 pipeline:

```
data/
  raw/
    alphaquery_iv/
    prices/
  processed/
    dow30/
    sp100/
    sp200/
    sp500_clean/
  configs/
    universes/
      dow30.yaml
      sp100.yaml
      sp200.yaml
      sp500_clean.yaml
  scripts/
    data/
      build_universe.py
      fetch_prices.py
      fetch_alphaquery_iv.py
      build_panel.py
    analysis/
      gnnhar_iv_scale_pipeline.py
  notebooks/
    gnnhar_iv_scale_colab.ipynb
  outputs/
    scale_experiment/
      dow30/
```

```
sp100/  
sp200/  
sp500_clean/
```

8.1 配置文件

每个 universe 一个 YAML:

```
name: sp100  
tickers_file: data/processed/sp100/tickers.txt  
start_date: "2021-01-26"  
end_date: "2026-01-26"  
rv_file: data/processed/sp100/merged_rv_data_filled.csv  
iv_file: data/processed/sp100/merged_iv_data_filled.csv  
returns_file: data/processed/sp100/daily_returns.csv  
split:  
  train: 0.70  
  valid: 0.15  
  test: 0.15  
graph:  
  method: glasso  
  target_density: 0.10  
models:  
  gnn_layers: [1, 2, 3]  
  hidden_grid: [16, 32, 64]  
  lr_grid: [0.0003, 0.001, 0.003]
```

8.2 主运行接口

主脚本接口应该变成:

```
python scripts/analysis/gnnhar_iv_scale_pipeline.py \  
  --config configs/universes/sp100.yaml \  
  --output-dir /content/drive/MyDrive/GNNHAR-colab-runs/outputs/scale_experiment/sp100 \  
  \  
  --tune-gnn \  
  --mcs-bootstrap 300 \  
  --seed 20260602
```

当前 `gnnhar_iv_pipeline.py` 已经有大部分模型逻辑。需要改的地方主要是:

- `load_panel()` 从固定 Dow 30 文件名改成读取 config。
- `glasso_adjacency()` 增加 target density 或固定 alpha 的选项。
- `evaluate_runs()` 增加 universe/name 字段, 方便横向合并。
- 新增 `scale_summary.csv`, 把不同 N 的 loss ratio 和 graph gain 汇总到一张表。

9 Colab 运行方案

Colab notebook 分成六个 cell 就够:

1. mount Google Drive。
2. clone GitHub branch。
3. install requirements。
4. 检查每个 universe 的三张数据宽表。
5. 按 universe 循环运行 pipeline。
6. 合并结果, 生成 scale-effect 表和图。

核心命令:

```
from google.colab import drive
drive.mount('/content/drive')

!git clone --depth 1 --branch 2026-06-01 https://github.com/easyg1lder/GNNHAR.git /
  content/GNNHAR
%cd /content/GNNHAR

!pip install -q numpy pandas scikit-learn scipy matplotlib torch pyyaml

universes = ["dow30", "sp100", "sp200", "sp500_clean"]
for u in universes:
    !python scripts/analysis/gnnhar_iv_scale_pipeline.py \
      --config configs/universes/{u}.yaml \
      --output-dir /content/drive/MyDrive/GNNHAR-colab-runs/outputs/scale_experiment/{u} \
      --tune-gnn \
      --mcs-bootstrap 300 \
      --seed 20260602

!python scripts/analysis/summarize_scale_experiment.py \
  --input-root /content/drive/MyDrive/GNNHAR-colab-runs/outputs/scale_experiment \
  --output-dir /content/drive/MyDrive/GNNHAR-colab-runs/outputs/scale_experiment/
  summary
```

S&P 500 的 GNNHAR3L-IV 可能比较慢。建议先跑 smoke test:

```
python scripts/analysis/gnnhar_iv_scale_pipeline.py \
  --config configs/universes/sp100.yaml \
  --output-dir outputs/scale_experiment/sp100_smoke \
  --fast \
  --epochs 40
```

确认数据、维度、loss 正常后再跑 full run。

10 最终论文表格

主表建议按 universe 展开：

Universe	N	HAR QLIKE	GHAR QLIKE	GNNHAR3L QLIKE	GHAR-IV QLIKE	GNN
Dow 30	30					
S&P 100	100					
S&P 200	200					
S&P 500-clean	N_{clean}					

第二张表直接报告增益：

Universe	G_N^{linear}	G_N^{gnn}	G_N^{iv}	G_N^{gnn-iv}
Dow 30				
S&P 100				
S&P 200				
S&P 500-clean				

其中

$$G_N^{iv} = 1 - \frac{L(HAR-IV_N)}{L(HAR_N)}.$$

11 解释逻辑

如果结果是：

$$G_{30}^{gnn-iv} \leq 0 \quad \text{但} \quad G_{100}^{gnn-iv} > 0,$$

你的解释会很强：Dow 30 过小，线性 GHAR 已经足够；更大图里，多跳和非线性 spillover 才有空间发挥。

如果结果是：

$$G_N^{linear} > 0 \quad \text{但} \quad G_N^{gnn} \leq 0$$

即使 $N = 500$ ，那说明图结构有用，但 GNNHAR 不一定比线性 GHAR 稳定，论文就应该强调 GHAR-IV 是更稳健的工程方案。

如果 random graph 在大 N 下也接近 GLASSO，说明模型可能只是受益于 cross-sectional smoothing 或参数扩张，不应声称“金融网络被识别出来”。这时要依赖 GLASSO vs random 的 placebo 结果来保护论文解释。

12 最小可执行路线

建议分三阶段：

1. 数据阶段：先完成 S&P 100 的三张宽表，完全复用现有 pipeline 跑通。
2. 代码阶段：把 `gnnhar_iv_pipeline.py` 参数化成 config-driven pipeline。
3. 论文阶段：补一节 “Network Scale Experiment”，把 Dow 30、S&P 100、S&P 200、S&P 500-clean 的结果放进同一张表。

最先跑 S&P 100，因为它最接近 Zhang et al. 的原始规模，也最能回答你现在的问题；S&P 500 是加分项，但数据完整性和 Colab 运行成本会更高。